04

Wissen abrufen

Alles in den Kontext · File-based Retrieval · RAG · Wissensbasis als Service

Cloud-native AI · B.Sc. Angewandte Informatik · HTWG Konstanz · WS 2026/27 · Grundlage: Huyen, AI Engineering, Kap. 6 (RAG) · Huß/Zonca, Generative AI on Kubernetes, Kap. 6 und 8

Lernziele

- ✓ Erklären, warum ein Modell Wissen im Kontext braucht, und drei Wege nennen, es dorthin zu bringen: alles mitgeben, Dateien durchsuchen, nach Bedeutung abrufen.
- ✓ Beschreiben, wie File-based Retrieval mit `grep`, `glob` und gezieltem Lesen funktioniert, und warum Coding-Agenten wie Claude Code ohne Index auskommen.
- ✓ Die RAG-Architektur mit Retriever und Generator, Indexierung und Abfrage erklären.
- ✓ Termbasiertes Retrieval (TF-IDF, BM25) und embeddingbasiertes Retrieval unterscheiden und ihre Stärken und Schwächen an Beispielen begründen.
- ✓ Chunking, hybride Suche, Reranking, Query Rewriting und Contextual Retrieval als Stellschrauben einordnen.
- ✓ Retrieval getrennt von der Antwort messen und begründet entscheiden, wann File-based Retrieval, wann RAG und wann beides passt.

Worum es geht

Der Campus-Assistent aus Übung 2 kennt genau vier Fakten: Bibliothek, Mensa, Prüfungsanmeldung und Kopierraum. Sie stehen in seinem System-Prompt. Fragt eine Studentin „Ich bin am Tag meiner Klausur krank. Was muss ich tun?“, hat das Modell zwei Möglichkeiten: Es gibt zu, dass es das nicht weiß, oder es erfindet eine plausible Antwort. Kleine Modelle wählen oft die zweite.

Was das Modell im Training gelernt hat, hilft hier nicht. Die Regeln einer bestimmten Hochschule standen nicht in den Trainingsdaten, und selbst wenn, wären sie vielleicht veraltet. Wissen, das ein Modell nicht gelernt hat, muss deshalb zur Laufzeit in seinen **Kontext**. Das kennt ihr aus VL 3: Was im Prompt steht, kann das Modell nutzen (In-Context Learning). Die Frage dieses Kapitels ist, **welches** Wissen in den Prompt kommt, wenn nicht alles hineinpasst oder hineinpassen soll.

Für die Übung hat der Campus-Assistent eine kleine Wissensbasis bekommen: 21 Markdown-Dateien von `bibliothek.md` bis `pruefungsordnung.md`, alle ausgedacht. Wir bringen ihr Wissen auf drei Wegen in den Kontext und messen sie gegeneinander:

1. **Alles in den Kontext:** Die ganze Wissensbasis steht in jedem Prompt. Keine Suche.
2. **File-based Retrieval:** Eine Textsuche wie `grep` findet die passenden Dateien, nur diese kommen in den Prompt.
3. **RAG (Retrieval-Augmented Generation):** Ein Embedding-Modell und eine Vektordatenbank finden die Abschnitte, deren Bedeutung der Frage am nächsten ist.

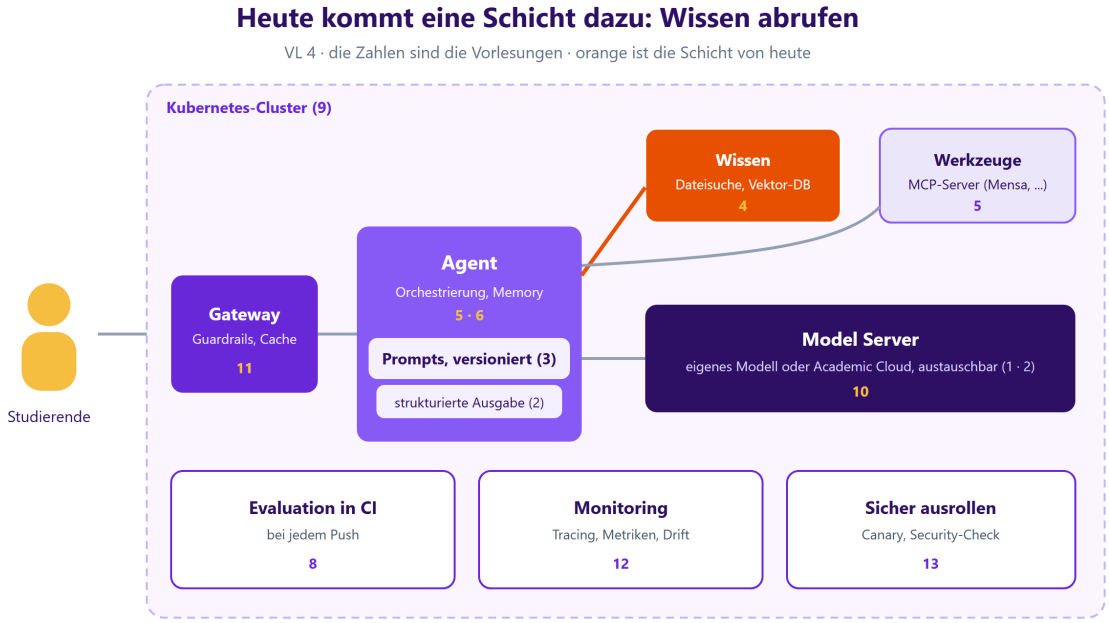


Abbildung 1: Die Zielarchitektur des Semesters. Heute kommt die Schicht „Wissen“ dazu, als eigener Service neben dem Agenten.

★ Huyen nennt das Zusammenstellen der passenden Informationen für eine Anfrage **Kontextkonstruktion** und vergleicht es mit dem Feature Engineering im klassischen Machine Learning: Das Modell kann nur so gut antworten, wie der Kontext es zulässt. RAG ist das bekannteste Muster dafür, aber nicht das einzige und nicht immer das beste.

Weg 0: Alles in den Kontext

Der einfachste Weg braucht keine Suche: Man gibt dem Modell bei jeder Frage die gesamte Wissensbasis mit. Unsere 21 Dateien sind zusammen etwa 1.600 Wörter lang, im Prompt rund 3.900 Tokens. Das passt in jedes aktuelle Kontextfenster.

Mit `llama3.2:3b` auf einem Laptop dauerte die erste Antwort so knapp eine Minute. Jede weitere dauerte im Schnitt 8 Sekunden. Der Grund ist der **Prompt-Cache**: Ollama merkt sich die Berechnung für einen Prompt-Anfang, der sich nicht ändert. Weil die Wissensbasis

immer gleich ist und vorne steht und nur die Frage am Ende wechselt, muss es nur die neuen Tokens rechnen. Kommerzielle APIs bieten dasselbe als **Prompt Caching** an, meist mit deutlich günstigeren Preisen für gecachte Tokens. Die Reihenfolge ist dabei entscheidend: erst das feste Wissen, dann die veränderliche Frage.

Bei 21 Dateien beantwortete diese Variante alle 14 Testfragen richtig, besser als jede Suche. Das ist kein Zufall. Anthropic empfiehlt: Ist die Wissensbasis kleiner als etwa 200.000 Tokens, also rund 500 Seiten, gebt sie einfach komplett mit. Huyen zitiert diese Faustregel und nennt trotzdem zwei Gründe, warum lange Kontextfenster RAG nicht überflüssig machen:

- **Daten wachsen schneller als Kontextfenster.** Eine Hochschule hat nicht 21 Dokumente, sondern Tausende, dazu Modulhandbücher, Protokolle und Mails.
- **Mehr Kontext ist nicht automatisch besser.** Jedes Token kostet Geld und Zeit, und das Modell findet die richtige Stelle in einem langen Kontext schlechter. Ihr kennt das als „Lost in the Middle“ aus VL 3. Anthropic spricht von **Context Rot**: Die Genauigkeit sinkt mit wachsendem Kontext nicht schlagartig, sondern schleichend.

> ★ Passt das Wissen in den Kontext und ändert es sich selten, ist „alles mitgeben“ die einfachste und oft beste Lösung. Suchen lohnt sich, wenn das Wissen zu groß, zu teuer oder zu oft geändert ist.

Teil 1: File-based Retrieval

Die Idee: suchen, wenn die Frage da ist

File-based Retrieval braucht keine Vorbereitung. Die Dokumente liegen als Dateien vor, und erst wenn eine Frage kommt, wird gesucht: mit `glob`, um Dateien nach Namen zu finden, mit `grep`, um Zeichenketten in Dateien zu finden, und mit einem gezielten Lesen der Treffer, ganz oder in Ausschnitten. Anthropic nennt das **Just-in-Time-Retrieval**: Das System hält nur leichte Verweise wie Dateipfade bereit und lädt die Inhalte erst bei Bedarf.

```
grep -ril "attest" wissen/            # welche Dateien enthalten "attest"?
grep -n -i "werktag" wissen/*.md      # mit Zeilennummer, ohne Groß/Klein
```

Woher kommen die Suchbegriffe? Im einfachsten Fall aus den Wörtern der Frage. Besser ist es, das Sprachmodell selbst die Suchbegriffe wählen zu lassen. Es kann Fachbegriffe ergänzen, die in der Frage nicht vorkommen, etwa „Attest“ oder „Prüfungsunfähigkeit“ für „krank am Klausurtag“. Ein Agent wiederholt das in einer Schleife: suchen, lesen, entscheiden, ob es reicht, und sonst mit neuen Begriffen weitersuchen.

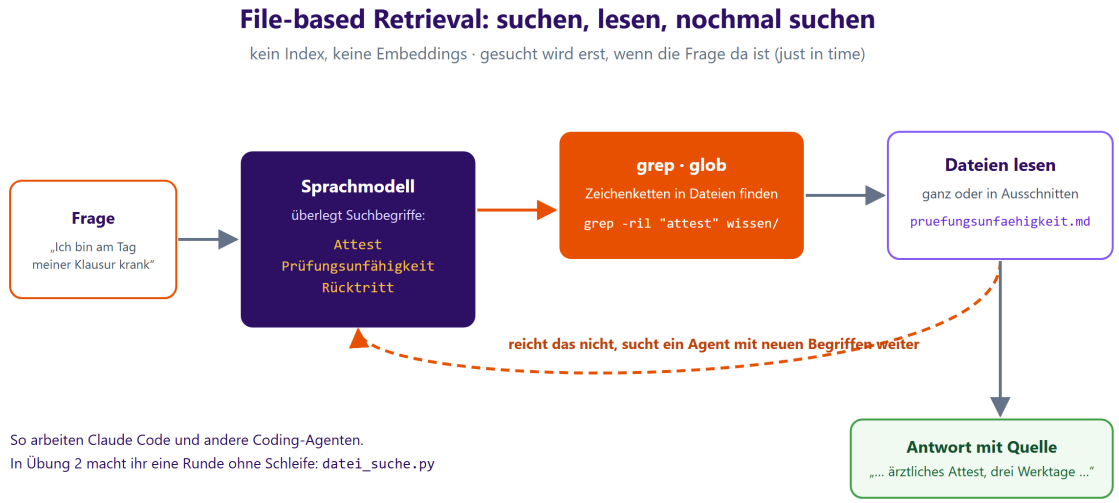


Abbildung 2: File-based Retrieval: Das Modell wählt Suchbegriffe, `grep` findet Dateien, das Modell liest sie. Ein Agent wiederholt die Runde bei Bedarf.

So arbeiten Coding-Agenten

Claude Code durchsucht ein Repository genau so. Beim Start lädt es eine Datei `CLAUDE.md` mit den wichtigsten Hinweisen zum Projekt vorab in den Kontext, alles andere sucht es zur Laufzeit mit `glob` und `grep`. Boris Cherny, der Claude Code bei Anthropic leitet, schrieb dazu im Februar 2026: Frühe Versionen hätten RAG mit einer lokalen Vektordatenbank genutzt, agentische Suche habe aber schnell besser funktioniert. Sie sei außerdem einfacher und habe weniger Probleme mit Sicherheit, Datenschutz, veralteten Indizes und Zuverlässigkeit. In einem Podcast räumte er ein, dass diese Entscheidung auf internen Erfahrungen beruhte, nicht auf einem Benchmark.

Bei Cursor verlief die Geschichte in zwei Schritten. Im November 2025 berichtete Cursor, dass eine semantische Suche über einen Embedding-Index die Antworten des Agenten im Mittel um 12,5 Prozent verbessere, je nach Modell zwischen 6,5 und 23,5 Prozent. Im Juli 2026 bestätigte ein Cursor-Mitarbeiter im offiziellen Forum, dass der Embedding-Index zugunsten einer grep-basierten Suche abgeschaltet wird. Ein weiterer Cursor-Mitarbeiter begründete das im selben Thread: Die Modelle seien in den letzten Monaten so gut darin geworden, mit `grep` zu suchen, dass die semantische Suche kaum noch etwas beitrug. Nebenbei entfällt damit ein Datenschutzproblem: Cursor berechnet keine Embeddings des Codes mehr und speichert sie nicht mehr auf seinen Servern.

Diese Geschichte zeigt, wovon File-based Retrieval lebt: von einem Modell, das gute Suchbegriffe wählt, die Treffer liest und bei Bedarf nachsucht.

Stärken

- **Exakt:** Eine Raumnummer wie `F-201`, ein Formular wie `PA-12` oder ein Funktionsname wird genau so gefunden, wie er geschrieben ist.

- **Immer aktuell:** Es gibt keinen Index, der veralten kann. Wer eine Datei ändert, findet die Änderung bei der nächsten Suche.
- **Keine Vorbereitung:** kein Chunking, kein Embedding-Modell, keine Vektordatenbank, kein zusätzlicher Service.
- **Nachvollziehbar:** Man sieht, welcher Begriff in welcher Zeile getroffen hat.
- **Rechte bleiben erhalten:** Die Suche läuft mit den Rechten des Nutzers auf seinen Dateien. Ein zentraler Index müsste die Zugriffsrechte selbst nachbilden.

Grenzen

- **Synonyme und Umschreibungen:** Wer „krank“ sucht, findet kein „Attest“, wer „Auto“ sucht, kein „Kfz“.
- **Wortteile:** `grep` findet Zeichenketten, keine Wörter. In unserer Wissensbasis trifft „tag“ in „Montag“, „tun“ in „Beratung“ und „raum“ in „Prüfungszeitraum“.
- **Bewertung:** Wer einfach Treffer zählt, lässt häufige Wörter gewinnen. Bei „Wer sitzt in Raum F-201?“ kommt „raum“ in vielen Dateien vor, die eine Datei mit „F-201“ landet nur auf Rang 4.
- **Kosten und Latenz:** Ein Agent, der fünf Runden sucht und liest, braucht fünf Modellaufrufe und viele Tokens.
- **Abhängigkeit vom Modell:** Unser `llama3.2:3b` schlug für „krank am Klausurtag“ die Begriffe „Krankheit, Arzt, Arztbescheinigung“ vor, aber nicht „Attest“. Die Suche fand die richtige Datei nicht.

> 💡 **Tipp:** In Übung 2 baut ihr eine Runde dieser Suche ohne Schleife: `datei_suche.py` lässt das Modell Suchbegriffe wählen, zählt die Treffer pro Datei und gibt die drei besten Dateien mit. Mit `--woerter` nimmt es stattdessen die Wörter der Frage.

Teil 2: RAG

Retriever und Generator

Retrieval-Augmented Generation, kurz RAG, verbindet eine Suche mit der Generierung. Das Muster stammt aus der Forschung zu Frage-Antwort-Systemen. Den Namen prägten Lewis et al. (2020). Ein RAG-System hat zwei Teile:

- Der **Retriever** findet zu einer Frage die passenden Textstücke.
- Der **Generator**, das Sprachmodell, formuliert daraus die Antwort.

Der Retriever arbeitet in zwei Abläufen. Bei der **Indexierung** werden die Dokumente vorab aufbereitet: in Abschnitte zerlegt (**Chunking**), für die Suche vorbereitet und gespeichert. Bei der **Abfrage** sucht der Retriever zu jeder Frage die besten Abschnitte.

Huß und Zonca betonen, dass die Indexierung offline und asynchron läuft und Abfragen nicht blockieren darf. Wie man indexiert, hängt davon ab, wie man später abfragen will.

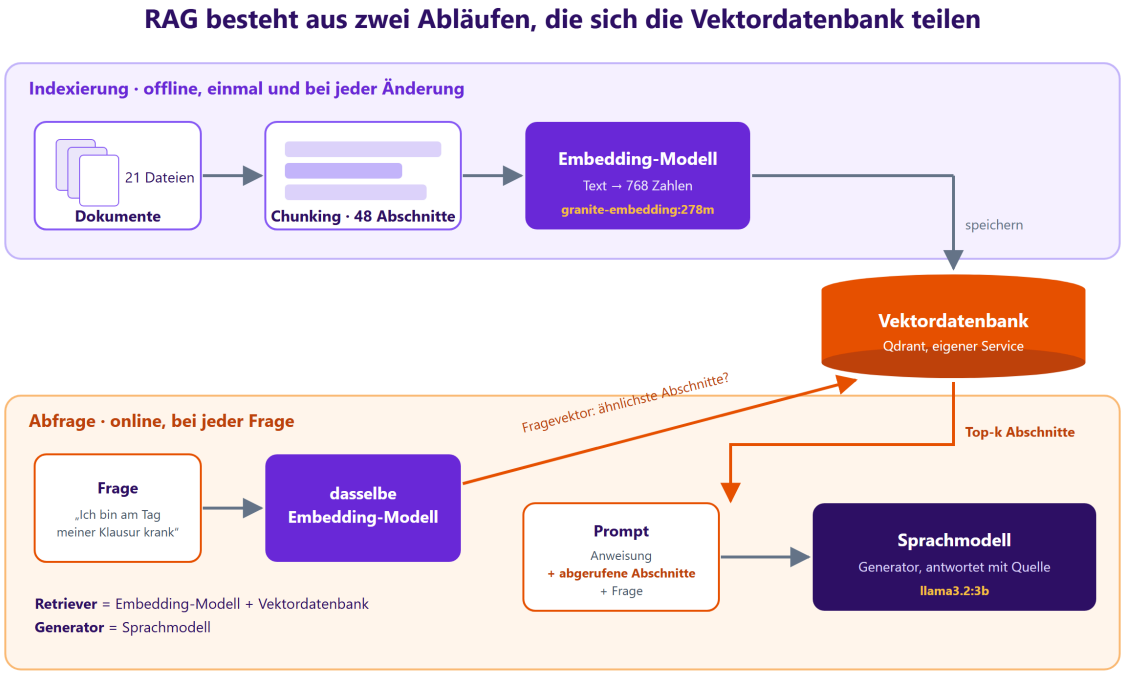


Abbildung 3: RAG mit Embeddings: Indexierung und Abfrage teilen sich das Embedding-Modell und die Vektordatenbank.

Huyen unterscheidet zwei Familien von Retrievern: termbasierte und embeddingbasierte.

Termbasiertes Retrieval: ein klügeres grep

Termbasiertes Retrieval sucht nach Wörtern, wie `grep`. Es macht aber zwei Dinge besser.

Erstens zählt es **ganze Wörter** statt Zeichenketten. Der Text wird in Terme zerlegt, kleingeschrieben und von Stoppwörtern wie „der“ und „und“ befreit. „tag“ trifft dann nicht mehr in „Montag“.

Zweitens **gewichtet** es die Terme. Die Termhäufigkeit (**TF**) zählt, wie oft ein Term in einem Dokument vorkommt. Die inverse Dokumenthäufigkeit (**IDF**) misst, wie selten er in der ganzen Sammlung ist:

$$\mathrm{IDF}(t) = \log \frac{N}{C(t)}$$

Dabei ist N die Zahl der Dokumente und $C(t)$ die Zahl der Dokumente, die den Term enthalten. Ein Term, der überall vorkommt, hat ein IDF nahe null und zählt kaum. Ein seltener Term zählt viel. **BM25** (Okapi BM25) ist die verbreitetste Variante von TF-IDF. Sie dämpft zusätzlich sehr häufige Wiederholungen und berücksichtigt die Länge eines Dokuments. Huyen nennt BM25 eine Messlatte, die schwer zu schlagen ist.

In unserem Beispiel „Wer sitzt in Raum F-201?“ bekommt „raum“ in BM25 ein Gewicht von 1,1, weil es in vielen Dateien steht, „f-201“ dagegen 2,7. Die richtige Datei rückt von Rang 4 auf Rang 1.

Damit die Suche schnell ist, legt man einen **invertierten Index** an: eine Tabelle, die zu jedem Term die Dokumente auflistet, in denen er vorkommt. Suchmaschinen wie Elasticsearch und OpenSearch arbeiten so.

Embeddingbasiertes Retrieval: Suche nach Bedeutung

Ein **Embedding** ist ein Vektor aus einigen hundert Zahlen, der die Bedeutung eines Textes beschreibt. Ein **Embedding-Modell** ist darauf trainiert, Texte mit ähnlicher Bedeutung auf ähnliche Vektoren abzubilden, auch wenn sie keine Wörter teilen. „Ich bin am Tag meiner Klausur krank“ und „ärztliches Attest, das die Prüfungsunfähigkeit am Prüfungstag bestätigt“ landen nah beieinander.

Wie ähnlich zwei Vektoren sind, misst die **Kosinus-Ähnlichkeit**: 1 heißt gleiche Richtung, 0 heißt unabhängig. Bei der Indexierung wird jeder Abschnitt eingebettet, bei der Abfrage die Frage, und zwar mit **demselben** Modell. Die Vektordatenbank liefert die k Abschnitte mit der höchsten Ähnlichkeit, die **Top-k**.

Bei wenigen tausend Vektoren kann man jede Ähnlichkeit ausrechnen. Bei Millionen Vektoren ist das zu langsam. Vektordatenbanken nutzen dann eine **approximative Nächste-Nachbarn-Suche** (ANN). Das verbreitetste Verfahren ist **HNSW**, ein mehrschichtiger Graph, in dem man sich von grob nach fein zu den nächsten Nachbarn bewegt. Man tauscht ein wenig Genauigkeit gegen sehr viel Geschwindigkeit. Bekannte Vektordatenbanken sind Qdrant, Milvus, Weaviate und pgvector als Erweiterung für PostgreSQL. In der Übung nutzen wir Qdrant.

Für unsere Wissensbasis haben wir vier Embedding-Modelle verglichen, die über Ollama lokal laufen. Gemessen wurde, ob die richtige Datei unter den vier ähnlichsten Abschnitten war:

Embedding-Modell	Größe	Sprachen	Richtige Datei in Top 4
`granite-embedding:278m`	562 MB	mehrsprachig	12 von 13
`embeddinggemma`	621 MB	mehrsprachig	12 von 13
`qwen3-embedding:0.6b`	639 MB	mehrsprachig	12 von 13
`nomic-embed-text`	274 MB	vor allem Englisch	10 von 13

Das englische Modell verlor genau die umschriebenen Fragen: krank, Auto, gestresst. Das Embedding-Modell muss zur Sprache und zur Domäne der Dokumente passen. Alle Modelle verfehlten „Raum F-201“: Ein Embedding fasst Bedeutung zusammen, eine Kennung aus Buchstaben und Ziffern hat kaum Bedeutung und geht dabei unter. Huyen nennt als Beispiel einen Fehlercode wie `EADDRNOTAVAIL (99)`, den eine Vektorsuche nicht zuverlässig findet. Und auch Embeddings sind nicht magisch: „Mir wird gerade alles zu viel“ fand die Psychosoziale Beratung nicht, obwohl dort wörtlich „wenn euch alles zu viel wird“ steht. Bei „Darf ich in der Klausur schummeln?“ standen Mensa und Rückmeldung vor § 17 Täuschung.

Termbasiert (BM25)	Embeddingbasiert
• schnell und billig, auch beim Indexieren • findet Kennungen, Namen und Codes exakt • funktioniert ohne Training „aus der Box“ • versteht keine Synonyme und Umschreibungen • wenig Stellschrauben zur Verbesserung	• findet Bedeutung, auch ohne gemeinsame Wörter • braucht ein Embedding-Modell und eine Vektordatenbank • Indexieren kostet Rechenzeit, Index kann veralten • verwischt Kennungen und seltene Begriffe • lässt sich durch Modellwahl und Finetuning verbessern

Retrieval verbessern

Chunking

Bevor Dokumente eingebettet werden, zerlegt man sie in Abschnitte. Ein Abschnitt über drei Themen bekommt einen Vektor, der zu keinem der Themen richtig passt. Zu kleine Abschnitte verlieren dagegen den Zusammenhang. Gängige Strategien:

- **feste Länge** in Zeichen, Wörtern oder Tokens, oft mit **Überlappung** (Overlap), damit kein Satz genau an der Grenze zerschnitten wird,
- **rekursiv** entlang der Struktur: erst an Überschriften, dann an Absätzen, dann an Sätzen,
- **spezialisiert** für Code, Tabellen oder Frage-Antwort-Paare.

In unserer Wissensbasis zerschneidet eine feste Länge von 160 Zeichen den Satz „Das Attest muss innerhalb von drei Werktagen nach dem Prüfungstermin …“. Ein Abschnitt kennt dann die Frist, ein anderer, wofür sie gilt. Mit 60 Zeichen Überlappung bleibt der Satz ganz, dafür entstehen sieben statt fünf Abschnitte. Zerlegt man an den Überschriften, entstehen 48 Abschnitte aus 21 Dateien. Huß und Zonca empfehlen für Handbücher und Richtlinien genau das: größere Abschnitte entlang der Überschriften, für FAQs eher einzelne Sätze. Huyen betont, dass es keine universell beste Größe gibt. Man muss experimentieren und messen.

> 💡 **Tipp:** Stellt jedem Abschnitt den Titel seines Dokuments voran. Der Abschnitt „Danach: Ihr werdet automatisch zum nächsten Termin angemeldet“ ist allein unverständlich, mit dem Titel „Rücktritt aus gesundheitlichen Gründen“ nicht.

Hybride Suche und Reranking

Weil termbasierte und embeddingbasierte Suche unterschiedliche Fehler machen, kombiniert man sie oft. Bei der **hybriden Suche** laufen beide parallel, und ihre Ranglisten werden zusammengeführt, meist mit **Reciprocal Rank Fusion** (RRF):

$$\text{Score}(D) = \sum_i \frac{1}{k + r_i(D)}$$

Dabei ist $r_i(D)$ der Rang des Dokuments in der Liste i, und k ist eine Konstante, typisch 60. Ein Dokument auf Rang 1 bei BM25 und Rang 3 bei der Vektorsuche bekommt $\frac{1}{61} + \frac{1}{63} \approx 0,0323$. Die Konstante sorgt dafür, dass ein einzelner Spitzenplatz nicht alles entscheidet.

Beim **Reranking** liefert eine schnelle Suche zuerst viele Kandidaten, etwa 50. Ein langsameres, genaueres Modell, oft ein **Cross-Encoder**, der Frage und Abschnitt gemeinsam liest, sortiert sie dann neu, und nur die besten gehen ins Sprachmodell.

Query Rewriting und Contextual Retrieval

Nutzerfragen sind oft unvollständig. Auf „Und wie ist das bei der Bachelorarbeit?“ findet keine Suche etwas, wenn sie den Gesprächsverlauf nicht kennt. Beim **Query Rewriting** formuliert ein Modell die Frage zuerst um, etwa zu „Wie lange ist die Bearbeitungszeit der Bachelorarbeit?“, und erst dann wird gesucht.

Contextual Retrieval setzt beim Index an. Ein Sprachmodell schreibt für jeden Abschnitt einen kurzen Kontextsatz, der sagt, woher er stammt und worum es geht, und dieser Satz wird vor dem Einbetten vorangestellt. Anthropic berichtet, dass die Fehlerrate beim Abruf der Top 20 dadurch von 5,7 auf 3,7 Prozent sinkt, mit zusätzlichem BM25 auf 2,9 und mit Reranking auf 1,9 Prozent, insgesamt um 67 Prozent.

Retrieval messen

Eine falsche Antwort kann zwei Ursachen haben: Die Suche hat die richtige Information nicht gefunden, oder das Modell hat sie nicht genutzt. Deshalb misst man beides getrennt. Huyen nennt für den Retriever zwei Maße:

- **Context Precision:** Welcher Anteil der abgerufenen Abschnitte ist relevant?
- **Context Recall:** Welcher Anteil der relevanten Abschnitte wurde abgerufen?

Recall ist in Produktion schwer zu messen, weil man dafür für jede Frage alle relevanten Dokumente kennen muss. Mit einem festen Satz von Testfragen, deren richtige Quelle bekannt ist, geht es. Genau das macht `vergleich_wissen.py` in der Übung: Die Spalte **Kontext** zeigt, ob die richtige Datei im Kontext stand, die Spalte **Antwort**, ob die Antwort stimmt.

RAG halluziniert trotzdem

RAG wird oft als Mittel gegen Halluzinationen verkauft. Eine Studie von Stanford RegLab und HAI (Magesh et al., 2024) prüfte juristische Recherchewerkzeuge, die mit „halluzinationsfreien“ Zitaten warben und alle auf RAG beruhen. Sie machten weniger Fehler als GPT-4 allein, lieferten aber trotzdem in 17 bis 33 Prozent der Fälle falsche oder nicht belegte Antworten. Fehler können an jeder Stelle entstehen: Die Suche findet ein unpassendes Dokument, das Modell liest es falsch, oder es mischt eigenes Wissen hinein.

Ein zweites Risiko ist die Quelle selbst. Im Mai 2024 empfahlen Googles AI Overviews, Käse mit Klebstoff auf der Pizza zu fixieren. Die Antwort stützte sich auf einen ironischen Forenbeitrag. Auf die Frage, wie viele Steine man essen solle, fand die Suche fast nur Satire und gab sie getreu wieder. Google nannte das selbst eine „Datenlücke“: Wo es kaum seriöse Inhalte gibt, gewinnt, was da ist.

> ⚠ **Achtung:** RAG macht ein Modell nicht wahrhaftig, sondern abhängig von seinen Quellen. Ein gutes System nennt die Quelle seiner Antwort und sagt „Dazu habe ich keine Information“, wenn die abgerufenen Abschnitte die Frage nicht beantworten. Unser Prompt `wissen.v1.yaml` verlangt genau das.

Die Wissensbasis als Service

Cloud-native wird es, sobald das Wissen ein eigener Baustein mit eigenem Lebenszyklus ist. In der Übung läuft Qdrant als eigener Container in Docker Compose, das Embedding-Modell läuft in Ollama, und der Index liegt in einem Volume. Huß und Zonca beschreiben, wie daraus in Kubernetes eigene Komponenten werden:

Komponente	Kubernetes-Baustein	Warum
Vektordatenbank	StatefulSet mit Persistent Volume, oft über einen Operator	hat Zustand, braucht stabilen Speicher und viel RAM für heiße Indizes
Embedding-Modell	eigenes Deployment oder Sidecar	muss für Indexierung und Abfrage identisch sein, skaliert mit der Last
Orchestrator (unser Python-Code)	zustandsloses Deployment mit Service	lässt sich beliebig vervielfachen
Indexierung (Ingestion)	Job oder CronJob, mit Ressourcengrenzen	läuft zeitgesteuert oder bei Änderungen und darf die Abfragen nicht ausbremsen

Daraus folgen drei Pflichten, die File-based Retrieval nicht hat:

- **Aktualität:** Jede Änderung an den Dokumenten muss in den Index. Wer `zerlege()` ändert oder das Embedding-Modell wechselt, muss alles neu indexieren.
- **Konsistenz:** Indexierung und Abfrage müssen dasselbe Embedding-Modell in derselben Version nutzen. Vektoren verschiedener Modelle sind nicht vergleichbar.
- **Zugriffsrechte:** Ein zentraler Index enthält alles. Ob eine Studentin ein Dokument sehen darf, muss die Suche selbst prüfen, zum Beispiel über Metadaten in jedem Punkt.

Das Kursbeispiel: drei Wege im Vergleich

Mit `llama3.2:3b`, `granite-embedding:278m` und Qdrant ergaben sich auf den 14 Testfragen der Übung diese Werte (ein Lauf, Temperature 0, Laptop ohne GPU):

Variante	Kontext richtig	Antwort richtig	Tokens (Ø)	Dauer (Ø)
alles in den Kontext	13 / 13	14 / 14	3.904	8,1 s
grep mit den Wörtern der Frage	7 / 13	9 / 14	682	9,4 s
Modell wählt Suchbegriffe, dann grep	8 / 13	10 / 14	602	11,0 s
RAG, ein Abschnitt pro Datei	12 / 13	12 / 14	891	11,6 s
RAG, an Überschriften zerlegt	12 / 13	13 / 14	555	8,6 s

Drei Beobachtungen:

- Bei 21 Dateien gewinnt „alles in den Kontext“ und braucht dafür siebenmal so viele Tokens. Bei einer kostenpflichtigen API ist das der Preis.
- Die Textsuche scheitert an den umschriebenen Fragen, RAG an der Kennung „F-201“. Das sind genau die erwarteten Schwächen der beiden Familien.
- Die Suche mit Modell ist kaum besser als die mit den Wörtern der Frage. Ein Modell mit 3 Milliarden Parametern wählt schlechte Suchbegriffe. Ein großes Modell wie in Claude Code wählt bessere und sucht bei Bedarf nach.

Wann was?

Es gibt keine Methode, die immer gewinnt. Die folgenden Fragen helfen bei der Entscheidung:

Wenn ...	... dann
das Wissen in den Kontext passt (Faustregel unter 200.000 Tokens) und sich selten ändert	alles mitgeben, mit Prompt Caching
Kennungen, Namen, Code und exakte Begriffe zählen und sich die Dateien oft ändern	File-based Retrieval, am besten mit einem starken Modell, das nachsucht
Nutzer in eigenen Worten fragen und die Wissensbasis groß ist	RAG mit Embeddings
beides vorkommt, wie fast immer in der Praxis	hybride Suche: BM25 und Embeddings, dazu Reranking
Antworten nachprüfbar sein müssen	Quelle mit ausgeben und „keine Information“ erlauben, egal welche Methode

Anthropic fasst die Haltung als Grundregel zusammen: Tut das Einfachste, das funktioniert. Für eine kleine Wissensbasis ist das gar keine Suche. Für Code ist es oft `grep`. Für Tausende Dokumente in natürlicher Sprache ist es RAG, in Produktion meist hybrid.

Die Vorlesung heute und Übung 2

In der Vorlesung probiert ihr beide Suchen in der **Retrieval-Spielwiese** aus: dieselbe Frage einmal mit `grep` beziehungsweise BM25 und einmal mit Embeddings, eine Karte der Abschnitte nach Bedeutung und ein Regler für das Chunking. In den letzten 30 Minuten arbeitet ihr im **Retrieval-Labor**.

In der Übung bearbeitet ihr **Teil 4 von Übung 2**: alles in den Kontext, Dateisuche mit `datei_suche.py`, RAG mit `rag.py` und Qdrant, Vergleich mit `vergleich_wissen.py` und eine eigene, gemessene Verbesserung. Abgabe ist am Montag, 02.11., um 23:59.

Typische Missverständnisse

⚠ **Achtung:**

- „RAG ist immer besser als eine einfache Suche.“ Bei Kennungen, Code und kleinen Wissensbasen gewinnt oft die einfache Suche oder gar keine.
- „Mit großen Kontextfenstern braucht man kein Retrieval mehr.“ Daten wachsen schneller, jedes Token kostet, und lange Kontexte werden schlechter genutzt.
- „Embeddings verstehen alles.“ Sie verwischen Kennungen und hängen von Sprache und Training des Embedding-Modells ab.
- „Wenn die richtige Datei im Kontext steht, stimmt die Antwort.“ Das Modell kann sie trotzdem falsch lesen. Deshalb misst man Kontext und Antwort getrennt.
- „Der Index ist einmal gebaut und fertig.“ Jede Änderung an Dokumenten, Chunking oder Embedding-Modell verlangt eine neue Indexierung.
- „RAG verhindert Halluzinationen.“ Es senkt sie, aber Studien messen weiterhin Fehlerraten von 17 bis 33 Prozent bei Produkten, die mit dem Gegenteil werben.

Zusammenfassung

★ Wissen, das ein Modell nicht gelernt hat, muss zur Laufzeit in den Kontext. Passt es hinein, ist „alles mitgeben“ mit Prompt Caching der einfachste Weg. Sonst muss gesucht werden. File-based Retrieval sucht erst, wenn die Frage da ist, mit `glob`, `grep` und gezieltem Lesen, ohne Index. Es ist exakt, immer aktuell und einfach, scheitert aber an Synonymen und lebt von einem Modell, das gute Suchbegriffe wählt. RAG trennt Retriever und Generator und indexiert die Dokumente vorab. Termbasiertes Retrieval wie BM25 gewichtet seltene Wörter und findet Kennungen exakt. Embeddingbasiertes Retrieval findet Bedeutung, auch ohne gemeinsame Wörter, verwischt aber Kennungen und hängt vom Embedding-Modell ab. Chunking, hybride Suche, Reranking, Query Rewriting und Contextual Retrieval sind die wichtigsten Stellschrauben. Retrieval misst man getrennt von der Antwort. Als Service braucht eine Wissensbasis eine Vektordatenbank mit Zustand, ein konsistentes Embedding-Modell und eine Indexierung, die aktuell bleibt.

Kontrollfragen

1. Eine Hochschule hat 40 FAQ-Seiten mit zusammen etwa 30.000 Tokens, die sich einmal im Semester ändern. Ein Team plant RAG mit Vektordatenbank. Was schlagt ihr stattdessen vor, und was müsst ihr bei der Reihenfolge im Prompt beachten?
2. Ein Support-Bot findet den Artikel zu Fehlercode `E-4711` nicht, obwohl er indexiert ist. Der Bot nutzt nur Embeddings. Erklärt die Ursache und nennt zwei Gegenmaßnahmen.
3. Ein Dokument steht bei BM25 auf Rang 2 und bei der Vektorsuche auf Rang 5. Berechnet seinen RRF-Score mit $k = 60$ und erklärt, wozu die Konstante dient.
4. Euer RAG-System beantwortet eine Frage falsch. Wie findet ihr heraus, ob die Suche oder das Modell schuld ist?
5. Ein Team wechselt das Embedding-Modell, ohne neu zu indexieren. Was passiert, und warum merkt man es vielleicht nicht sofort?
6. Claude Code nutzt keinen Embedding-Index, Cursor hat seinen 2025 eingeführt und 2026 wieder abgeschaltet. Nennt zwei Gründe, die für File-based Retrieval bei Code sprechen, und eine Situation, in der ihr trotzdem RAG wählen würdet.

Quellen

- Chip Huyen: *AI Engineering. Building Applications with Foundation Models.* O'Reilly, 2025, Kapitel 6, Abschnitt RAG.
- Roland Huß, Daniele Zonca: *Generative AI on Kubernetes.* O'Reilly, 2026, Kapitel 6 (Prompt and Context Engineering) und Kapitel 8 (Retrieval-Augmented Generation).
- Anthropic: *Effective context engineering for AI agents.* 29.09.2025.
- Anthropic: *Introducing Contextual Retrieval.* 19.09.2024.
- Patrick Lewis et al.: *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS, 2020.
- Stephen Robertson, Hugo Zaragoza: *The Probabilistic Relevance Framework: BM25 and Beyond.* 2009.
- Gordon V. Cormack et al.: *Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods.* SIGIR, 2009.
- Varun Magesh et al.: *Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools.* 2024, und Stanford HAI, 23.05.2024.
- Fälle: Boris Cherny, Post auf X, 01.02.2026 · Latent Space Podcast: *Claude Code: Anthropic's Agent in Your Terminal*, 2025 · Cursor: *Improving agent with semantic search*, 06.11.2025 · Cursor Community Forum, Thema „What do you think about Cursor removing the codebase indexing settings?“, 22.07.2026 · Elizabeth Reid, Google: *AI Overviews: About last week*, 30.05.2024.